Popularne strukture dubokih neuronskih mreža i *Transfer learning*

Sadržaj

- **Popularne strukture dubokih neuronskih mreža**
- ***Transfer learning***
- **Primjeri i zadaci**

Popularne strukture dubokih neuronskih mreža

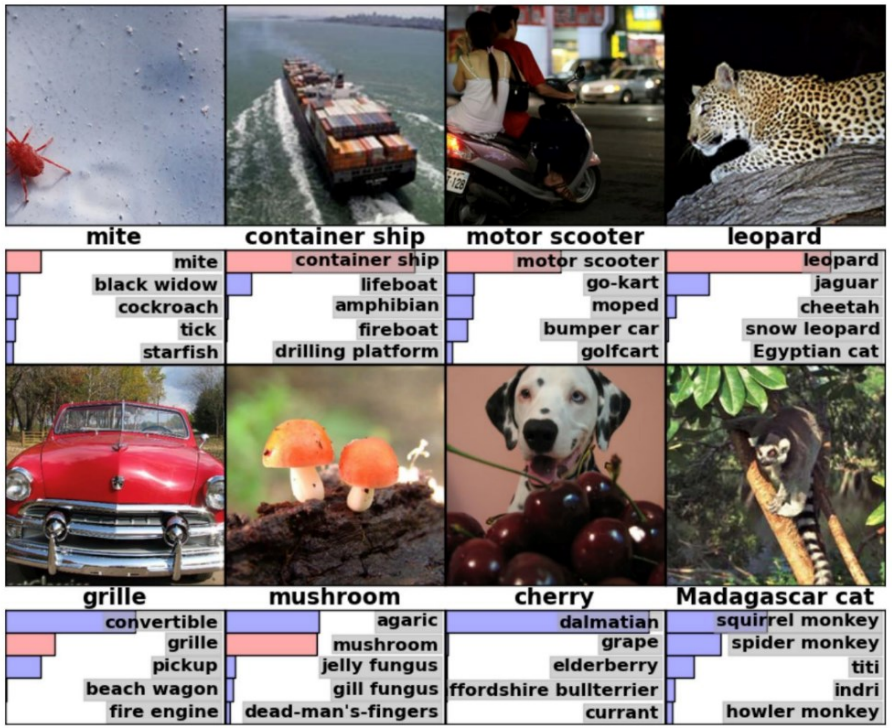
ImageNet challenge

- Pokrenuto 2009. godine

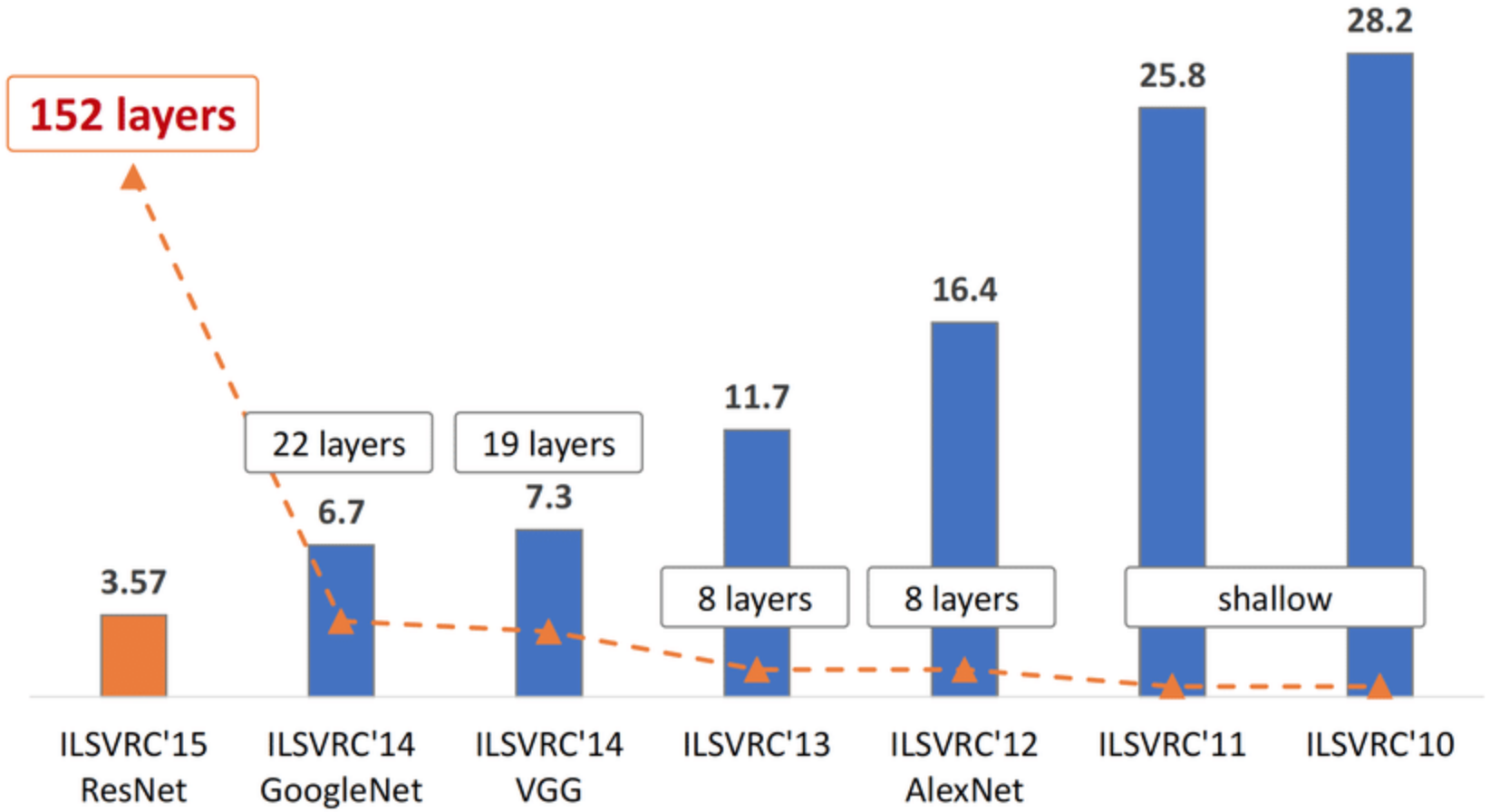
ImageNet Challenge

IMAGENET

- 1,000 object classes (categories).
- Images:
  - 1.2 M train
  - 100k test.



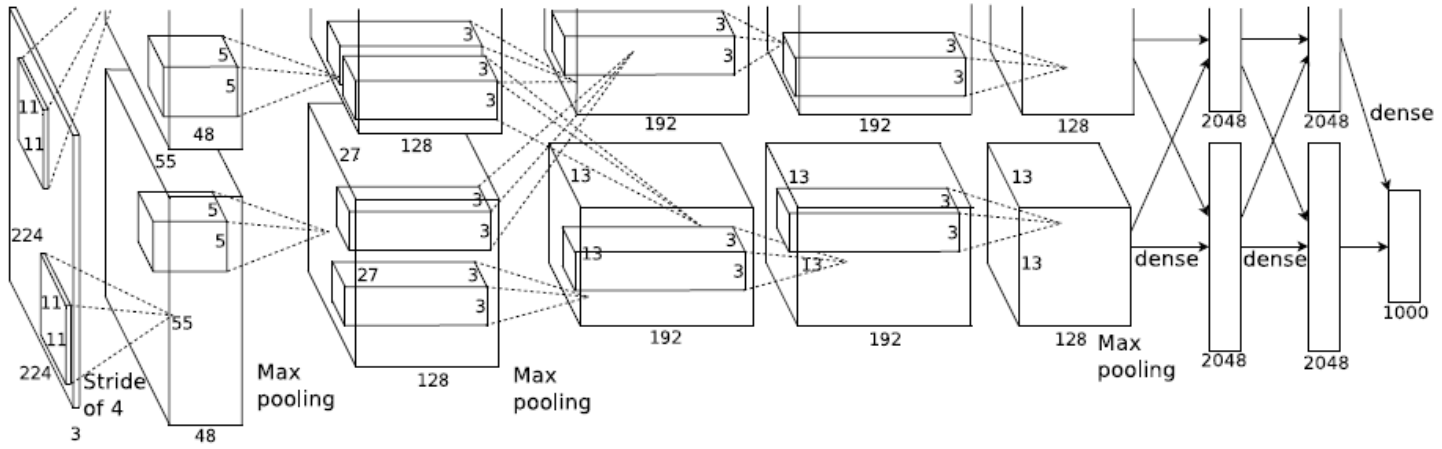
ImageNet challenge – pobjednici



Source

AlexNet (2012.)

- Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks, 2012.



Details/Retrospectives:
- first use of ReLU
- used Norm layers (not common anymore)
- heavy data augmentation
- dropout 0.5
- batch size 128
- SGD Momentum 0.9
- Learning rate 1e-2, reduced by 10 manually when val accuracy plateaus
- L2 weight decay 5e-4
- 7 CNN ensemble: 18.2% -> 15.4%

Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

VGG (2014.)

Karen Simonyan, Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014.

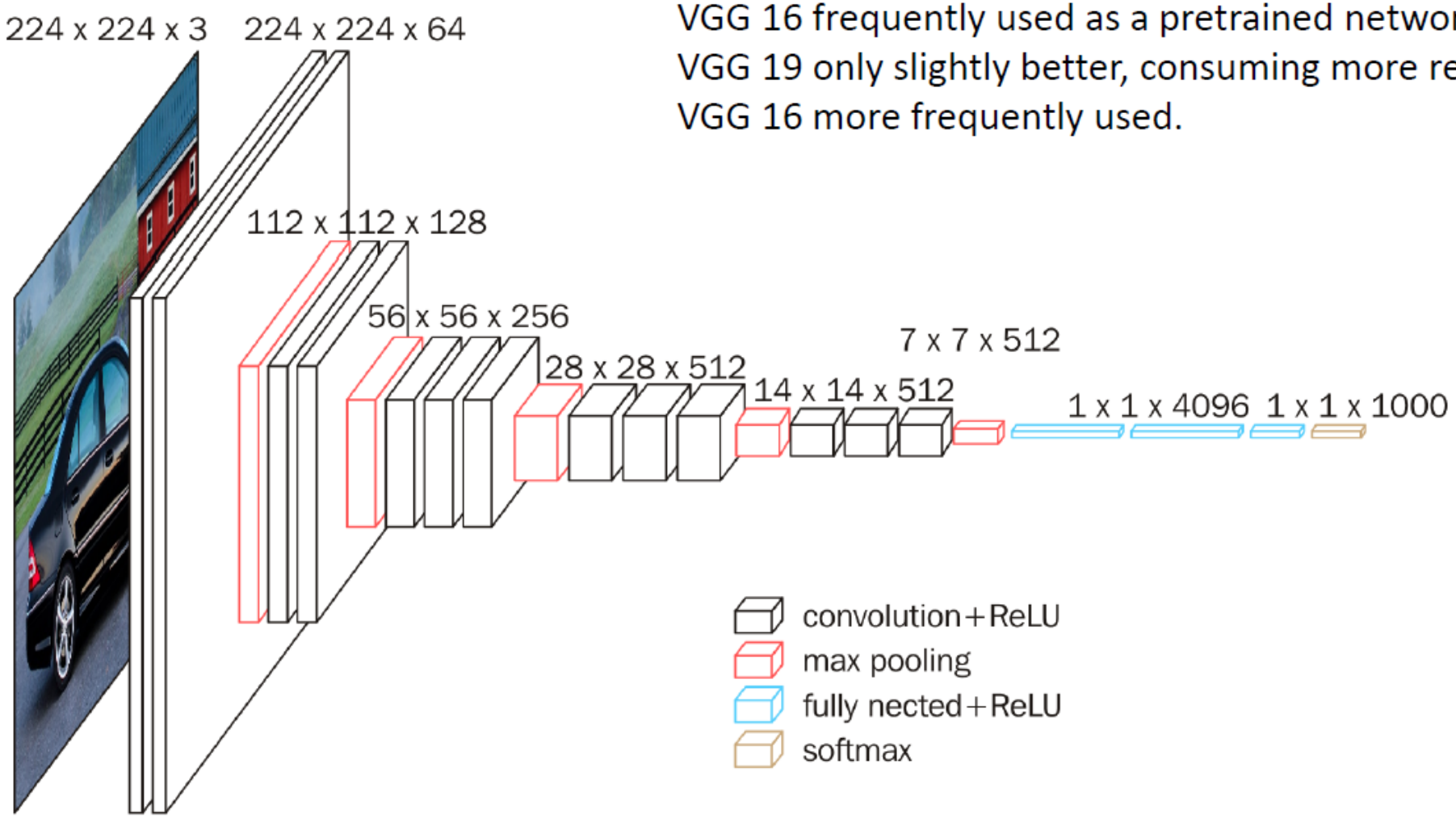
- Manji filtri (3x3)
- Više slojeva (dublja mreža)
- ReLU
- Sličan proces učenja kao kod AlexNet
- 7.3% top 5 error in ILSVRC 2015
- Best model VGG19

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

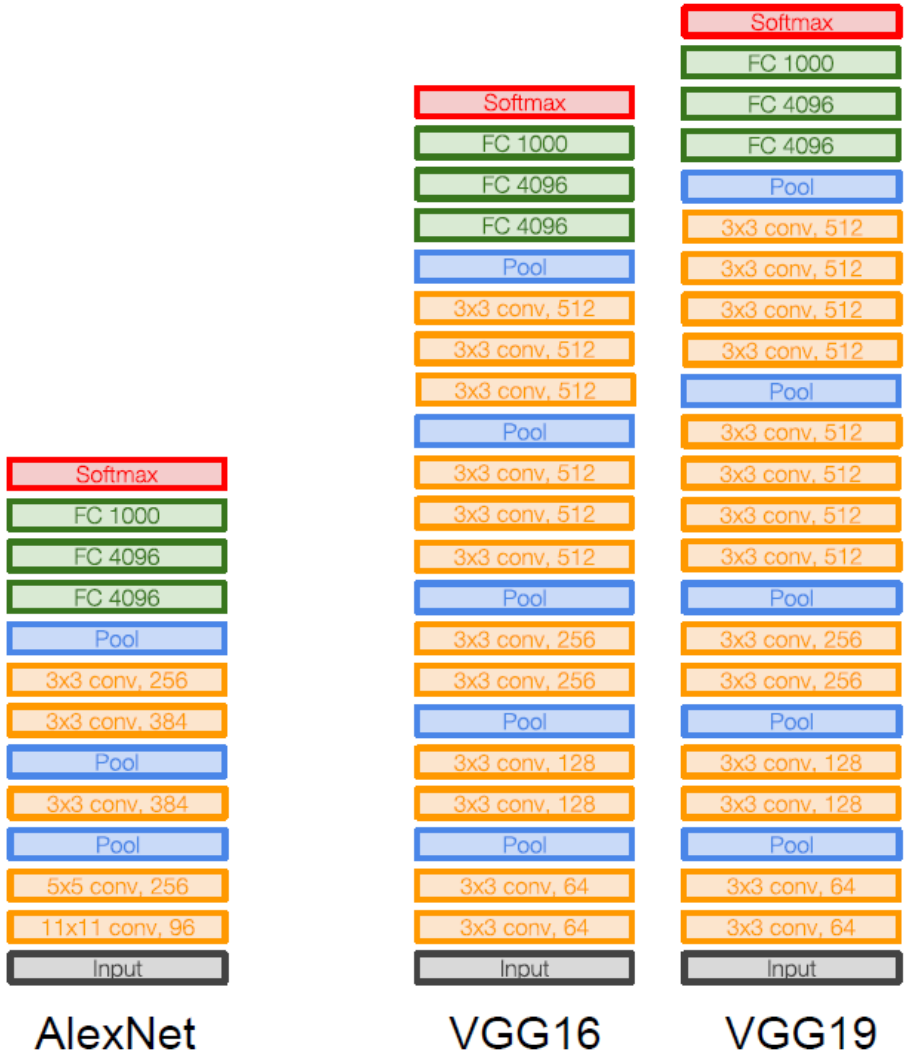
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG16



VGG 16 frequently used as a pretrained network.
VGG 19 only slightly better, consuming more resources.
VGG 16 more frequently used.

VGG vs AlexNet



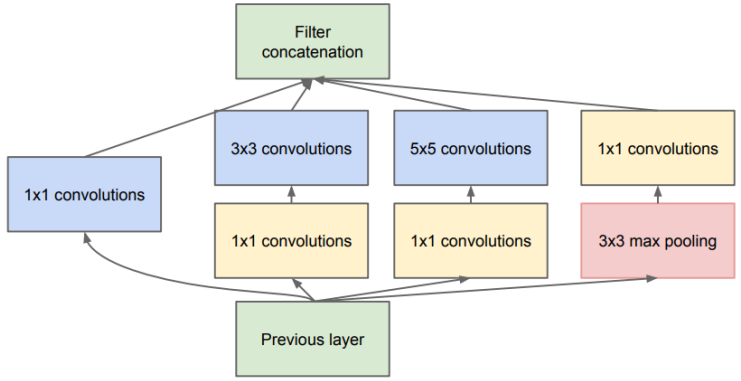
Zašto 3x3 filtri?

„It is easy to see that a stack of two 3×3 conv. layers (without spatial pooling in between) has an effective receptive field of 5×5; three such layers have a 7 × 7 effective receptive field. So what have we gained by using, for instance, a stack of three 3×3 conv. layers instead of a single 7×7 layer? First, we incorporate three non-linear rectification layers instead of a single one, which makes the decision function more discriminative. Second, we decrease the number of parameters: assuming that both the input and the output of a three-layer 3 × 3 convolution stack has C channels…”

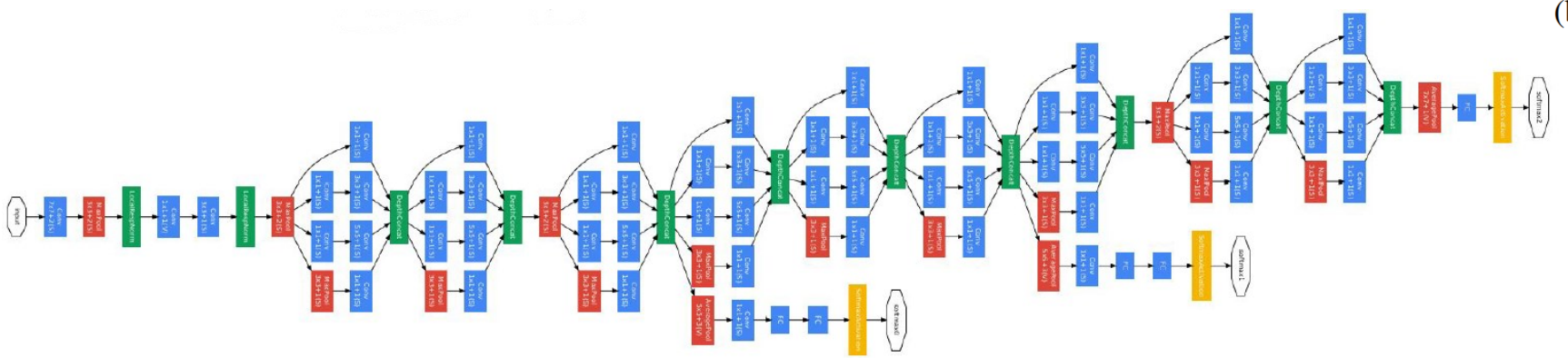
Kako unaprijediti učenje CNN?

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich, Going Deeper with Convolutions, 2014.

- Nastavlja se trend dubljih mreža s naglaskom na računalnu učinkovitost
- GoogLeNet (Inception):
  - učinkoviti Inception moduli
  - nema FC slojeva
  - Samo 5 milijuna parametera (12x manje nego AlexNet)
  - 6.7% top 5 error ILSVRC 2014



(b) Inception module with dimension reductions



Povećavanje dubine mreže

- Problem kada se uče izrazito duboke mreže na klasičan način
- Očekujemo da se poveća preciznost s povećanjem dubine mreže na skupu za učenje

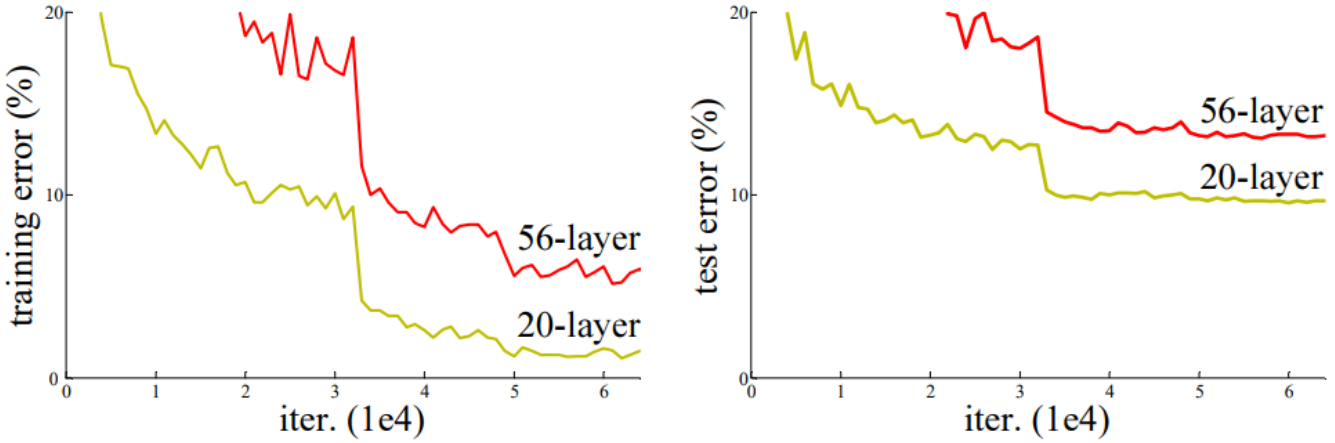
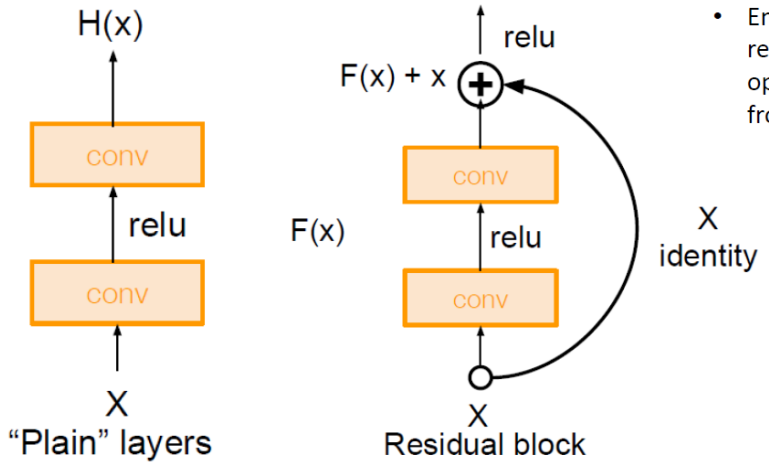


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

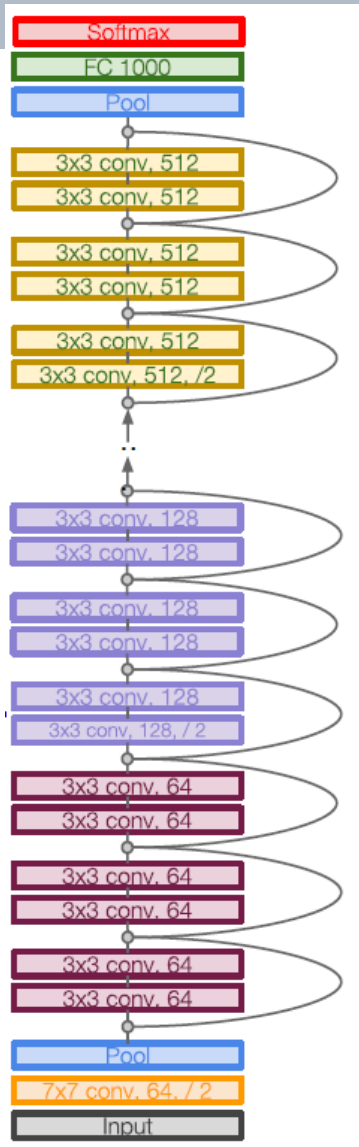
Ovo nije problem pretjeranog usklađivanja!

ResNet (2015.)

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, Deep Residual Learning for Image Recognition, 2015.
  - Revolucija u dubini mreže
  - Prvo mjesto u 5 različitih natjecanja (ImageNet i COCO)
  - Koristi residual blokove; svaki ima dva 3x3 konvolucijska sloja



- Empirical evidence showing that residual networks are easier to optimize, and can gain accuracy from increased depth.



ResNet (2015.)

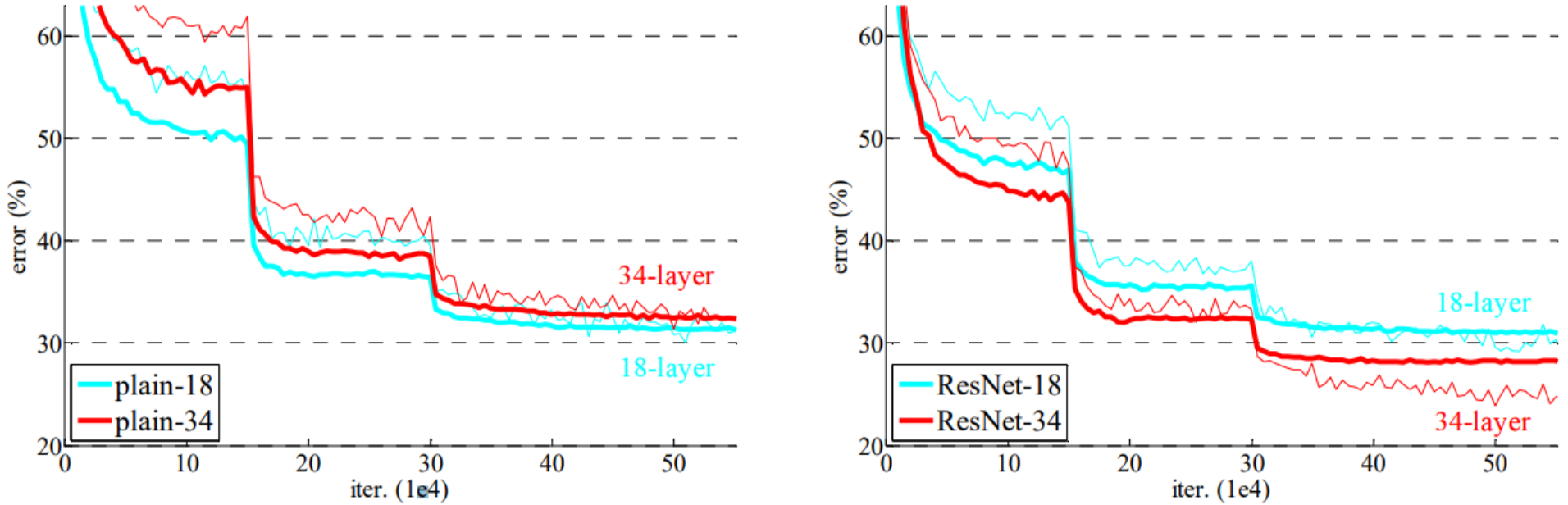


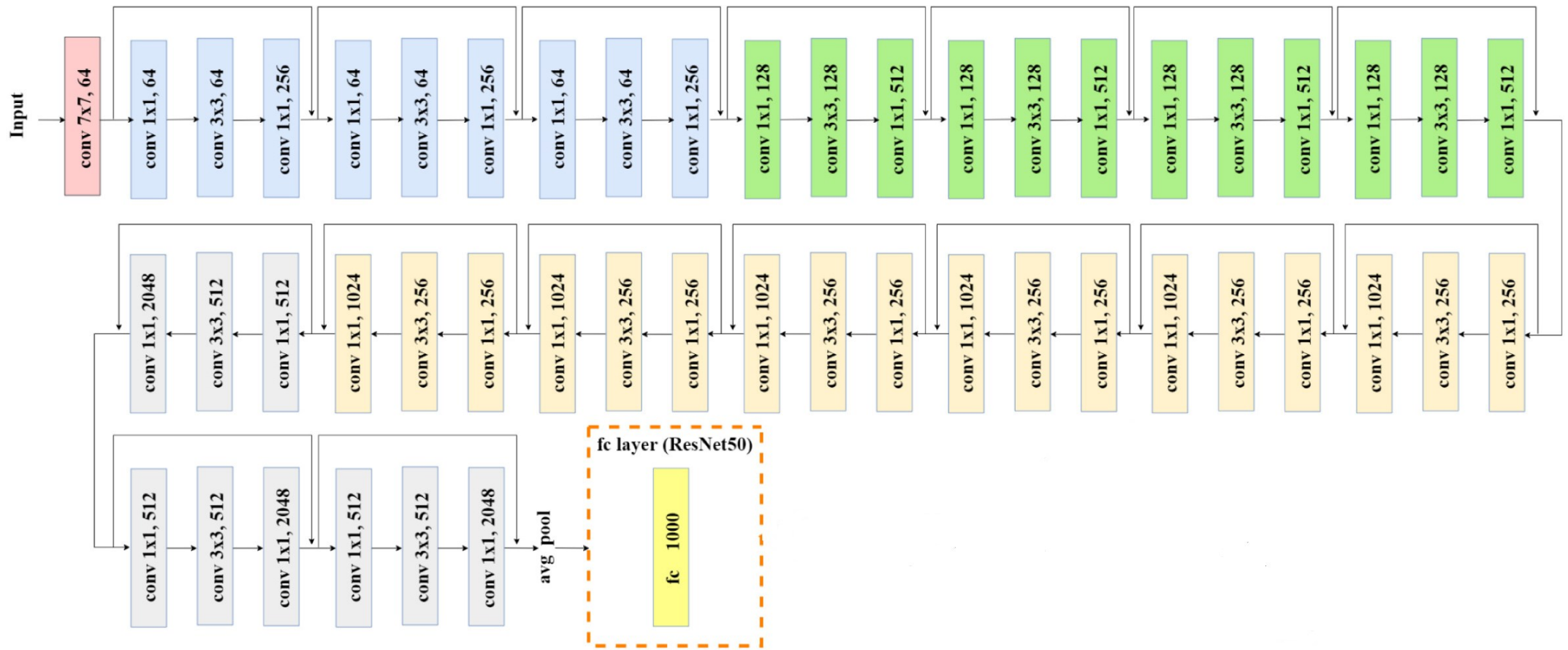
Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

Table 2. Top-1 error (%, 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

ResNet 50

- Vrlo popularna mreža



Usporedba računalnih zahtjeva i performansi

- A. Canziani, E. Culurciello, A. Paszke, AN ANALYSIS OF DEEP NEURAL NETWORK MODELS FOR PRACTICAL APPLICATIONS, 2017.

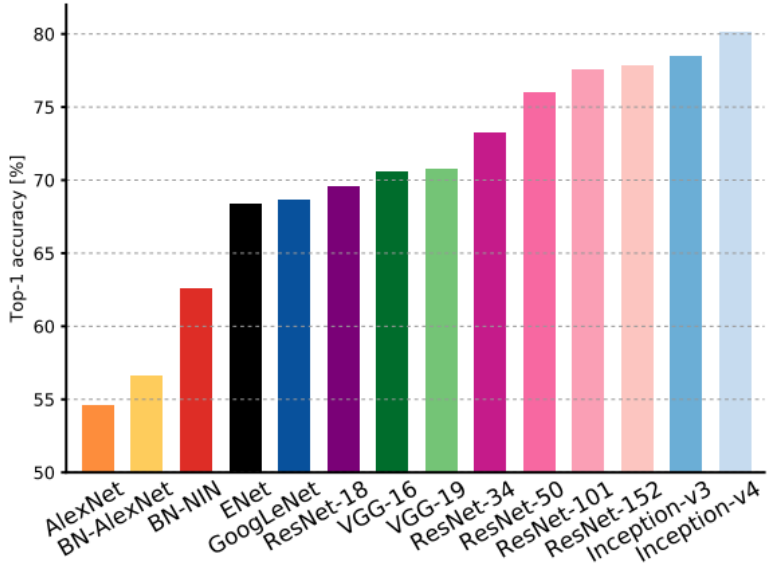


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

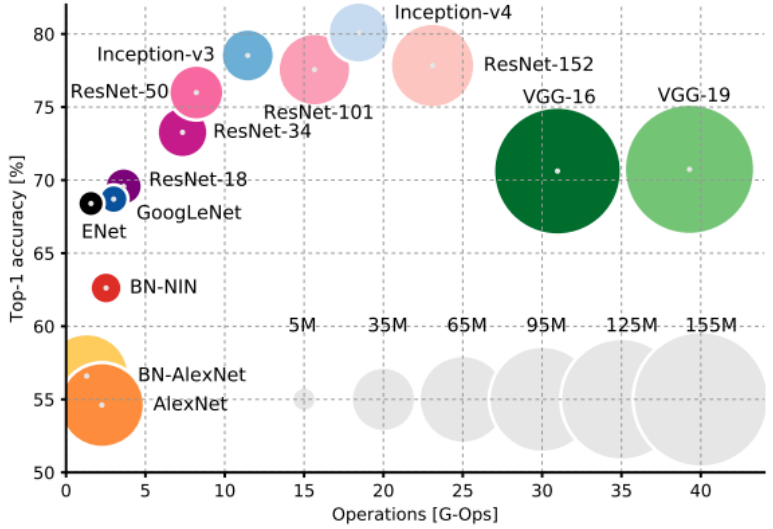


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same y-axis, and the grey dots highlight the centre of the blobs.

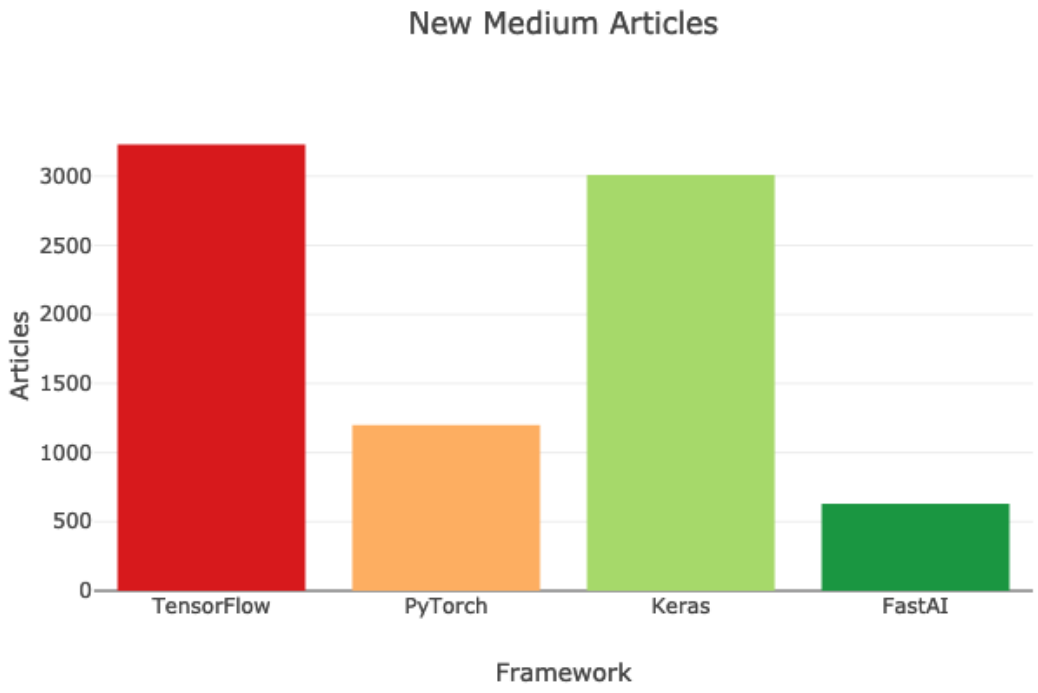
Mnoštvo različitih arhitektura...

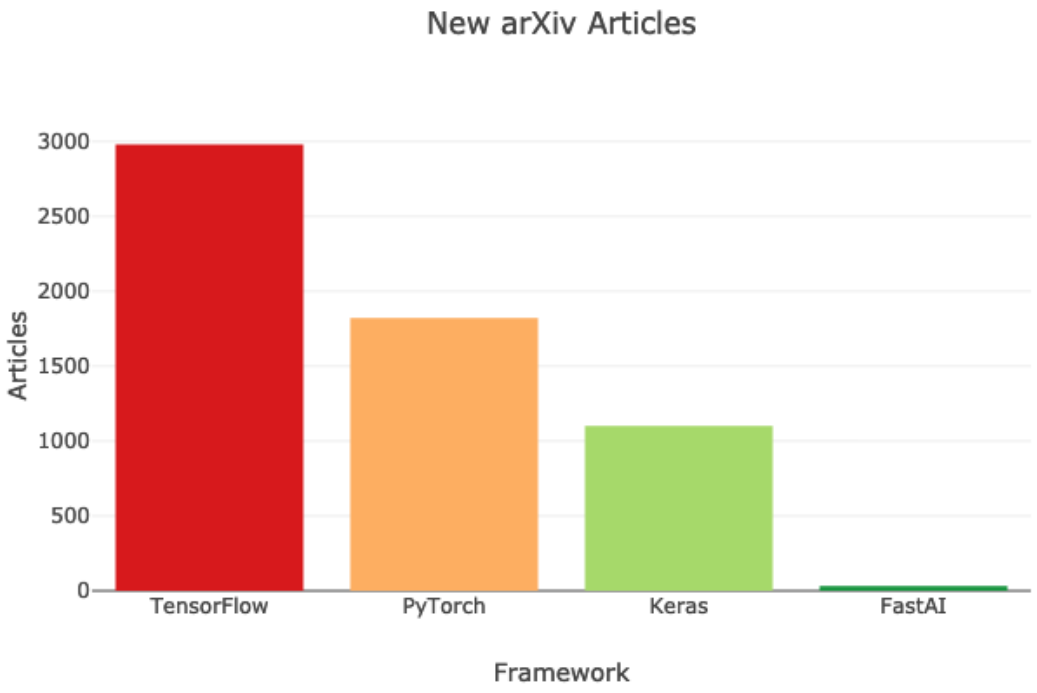
- Međutim, u posljednjih nekoliko godina pojavilo se više arhitektura neuronskih mreža:
  - MobileNets
  - DenseNets
  - EfficientNet
  - …
- Alati za optimizaciju neuronskih mreža, npr. Tensor RT (npr. ubrzanje za 36x)

Biblioteke za duboko učenje

- trenutno su vrlo popularni: TensorFlow, Pytorch i high level API (Keras i FastAI)

Izvor





Transfer learning

Transfer learning

- CNN može imati milijune parametara
- Naš dostupni skup podataka (slika) nije uvijek velik
- Postavlja se pitanje može li se CNN naučiti na takvim skupovima podataka bez *overfittinga*?
- Jedan od načina da naučite CNN mrežu je prijenosno učenje (engl. *transfer leraning*) koje omogućuje izgradnju CNN mreže iako naš skup podataka nije jako velik
- Pretpostavimo da imamo dva dostupna skupa podataka
  - Prvi skup slika je u potpunosti označen, ima veliki broj slika
  - Drugi skup podataka slika je sličan, ali sadrži znatno manje slika
  - **Oznake ne moraju biti iste u oba skupa podataka!**
    - **npr. ImageNet ima 1000 klasa, a naš problem samo nekoliko klasa**

Transfer learning

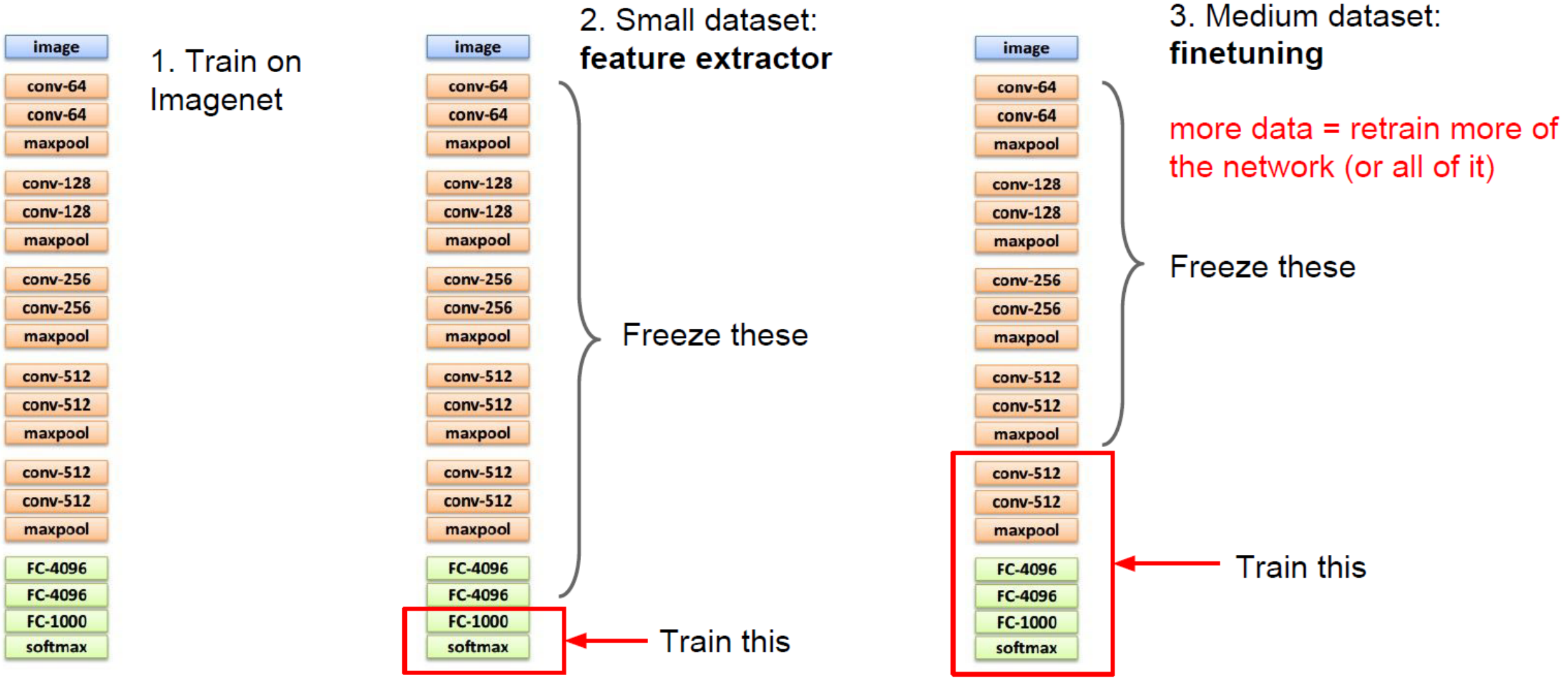
Imagenet: 1million

My dataset: 1,000

Model 1

Model 2

Transfer learning



Keras Applications

- Keras Applications su modeli dubokog učenja koji su dostupni uz unaprijed istrenirane težine
- Ovi se modeli mogu koristiti za predviđanje, izdvajanje značajki i fino podešavanje

https://keras.io/api/applications/

Available models

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4

Keras Applications

- Primjer ResNet50
- include_top: treba li uključiti potpuno povezani sloj na samom kraju mreže ili ne (onaj zadnji)
- weights: jedan od None (nasumična inicijalizacija), 'imagenet' (prethodna obuka na ImageNetu) ili putanja do datoteke s težinama koja se učitava
- input_shape: neobavezni *tuple*, samo se navodi ako je include_top False (inače ulaz mora biti oblika (224, 224, 3))

ResNet50 function

```
tf.keras.applications.ResNet50(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    **kwargs
)
```

Instantiates the ResNet50 architecture.

Keras *trainable*

- Slojevi i modeli imaju tri atributa težina
- weights: je popis svih varijabli težina sloja
- trainable_weights: je popis onih koji se trebaju ažurirati kako bi se smanjio *loss* tijekom treninga
- non_trainable_weights: popis je onih težina koje nisu namijenjene treniranju
- Ideja s prijenosnim učenjem je "zamrznuti" slojeve postavljanjem **trainable** parametra na False

Transfer learning - Keras *trainable*

- Učitati pre-trenirani model

```
base_model = keras.applications.Xception(
    weights='imagenet',  # Load weights pre-trained on ImageNet.
    input_shape=(150, 150, 3),
    include_top=False)  # Do not include the ImageNet classifier at the top.
```

- „Zamrznuti” težine

```
base_model.trainable = False
```

Transfer learning - Keras trainable

- dodati vlastite slojeve na bazni model

```
inputs = keras.Input(shape=(150, 150, 3))
# We make sure that the base_model is running in inference mode here,
# by passing `training=False`. This is important for fine-tuning, as you will
# learn in a few paragraphs.
x = base_model(inputs, training=False)
# Convert features of shape `base_model.output_shape[1:]` to vectors
x = keras.layers.GlobalAveragePooling2D()(x)
# A Dense classifier with a single unit (binary classification)
outputs = keras.layers.Dense(1)(x)
model = keras.Model(inputs, outputs)
```

Transfer learning - Keras trainable

- trenirati model na vlastitim podacima

```
model.compile(optimizer=keras.optimizers.Adam(),
              loss=keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=[keras.metrics.BinaryAccuracy()])
model.fit(new_dataset, epochs=20, callbacks=..., validation_data=...)
```

Transfer learning - Keras trainable

- opcija - provesti fino podešavanje parametara
- paziti da se ne dogodi overfit (ići s manjom stopom učenja)!

```
# Unfreeze the base model
base_model.trainable = True

# It's important to recompile your model after you make any changes
# to the `trainable` attribute of any inner layer, so that your changes
# are take into account
model.compile(optimizer=keras.optimizers.Adam(1e-5),  # Very low learning rate
              loss=keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=[keras.metrics.BinaryAccuracy()])

# Train end-to-end. Be careful to stop before you overfit!
model.fit(new_dataset, epochs=10, callbacks=..., validation_data=...)
```

Zadatak 1 – korištenje dostupnih popularnih dubokih neuronskih mreža

1. Učitaj jedan od postojećih modela u Keras-u (npr. Resnet50 ili MobileNetV2)
   - koristite opcije: `weights: „imagenet”, include_top = True`
2. učitajte proizvoljnu sliku (npr. sliku automobila, banane, stolice ...) pomoću Keras funkcije `load_img`
3. Napravite odgovarajuću predobradu učitane slike (`img_to_array`, `expand_dims`, `preprocess_input`) te izvršite predikciju pomoću odabranog modela.
4. Budući da se radi o mrežni naučenoj na ImageNet-u, pomoću funkcije `keras.applications.imagenet_utils.decode_predictions` pretvorite predikciju u odgovarajuću klasu
5. „igrajte” se s mrežom (Izvršite inferenciju nad više različitih slika i analizirajte rezultate)
6. Uočavate li razlike u točnosti klasifikacije između različitih mrežnih arhitektura?

Zadatak 2 – treniranje CNN mreže na CIFAR-10 skupu podataka

1. Proučite kod u skripti Zadatak_2.py
2. Pokrenite skriptu i pratite tijek treniranja mreže.
3. Komentirajte dobivene rezulate.

Zadatak 3 – Transfer learning

- Na temelju skripte iz prethodnog zadatka potrebno je učiniti transfer learning:
  - Učitajte unaprijed istreniranu mrežu (npr. MobileNetv2 na imagenet) bez završnog klasifikacijskog sloja
  - Zamrznuti težine bazne (pretrenirane) mreže
  - Prilagodite primjere (slike) odabranoj baznoj mreži
  - Dodati vlastiti klasifikacijski sloj prilagođen broju klasa u korištenom skupu podataka (CIFAR-10)
  - Trenirajte mrežu nekoliko epoha
  - Kakve rezultate postižete ovakvim postupkom u odnosu na prethodni zadatak? Što kada biste u oba slučaja imali 10x manje primjera?

Zadatak 4 – Transfer learning

- Opcionalno:
  - Provesti djelomični fine-tuning odmrzavanjem zadnjih konvolucijskih slojeva uz manju stopu učenja
  - Provesti potpuni fine-tuning cijele mreže uz vrlo malu stopu učenja